

Basic (a normal, exact test) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `chunking_unit_test_practice.py`: the chunker has no model inside, so it's the easiest place to learn pytest itself — `test_` functions, plain `assert`, and reading a failure — before any LLM output makes checking harder. **If not:** your first pytest test would be against a model's answer, and you couldn't tell whether a failure came from pytest, your check, or the model.

Copy Doc08's `chunking.py` into `practice/` first, unchanged. Run every version from inside `practice/` with `pytest chunking_unit_test_practice.py -v`. Read both depths — they're not "wrong, right," they're 2 real, valid ways to test the same function, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# practice/chunking_unit_test_practice.py
from chunking import chunk_by_paragraph

sample_text = "Paragraph one.\n\nParagraph two.\n\nParagraph three."

def test_chunk_by_paragraph():
    chunks = chunk_by_paragraph(sample_text)
    assert len(chunks) == 3
```

Expected output:

```
chunking_unit_test_practice.py::test_chunk_by_paragraph PASSED
```

This is a correct, minimal test — it just checks the count, not the content of each chunk.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — count plus content, and a second test for the single-paragraph case

Story: a count-only check still passes if the chunker scrambles the order or cuts text off. Checking the first and last chunk's text catches that, and a second, clearly named test covers the "no blank lines at all" input real documents often have. **If not:** a chunker that quietly drops the last sentence would pass, and your RAG answers would be missing facts with no failing test to say why.

```
# practice/chunking_unit_test_practice.py
from chunking import chunk_by_paragraph

sample_text = "Paragraph one.\n\nParagraph two.\n\nParagraph three."

def test_chunk_by_paragraph():
    chunks = chunk_by_paragraph(sample_text)
    assert len(chunks) == 3
    # why: content checks catch a scrambled order or cut-off text,
    # which a count check alone would miss
    assert chunks[0] == "Paragraph one."
    assert chunks[-1] == "Paragraph three."

# how: one test per case, named after the case –
# the name alone tells you what broke
def test_chunk_by_paragraph_single_paragraph():
    chunks = chunk_by_paragraph("Just one paragraph, no blank lines.")
    assert len(chunks) == 1
    assert chunks[0] == "Just one paragraph, no blank lines."
```

Expected output:

```
chunking_unit_test_practice.py::test_chunk_by_paragraph PASSED
chunking_unit_test_practice.py::test_chunk_by_paragraph_single_paragraph PASSED
```

Approach 2 — `pytest.mark.parametrize` to cover several inputs at once

Story: messy inputs (an empty file, extra blank lines, a file of only spaces) each deserve a check, but writing a new function for every one gets long fast. `parametrize` runs one test body over a table of cases. **If not:** you'd stop at two or three cases because each new one costs a whole function, and the empty-file case would never get tested.

```
# practice/chunking_unit_test_practice.py
import pytest
from chunking import chunk_by_paragraph

# how: pytest runs the test once per row, filling in text and
# expected_count from that row – each row is its own pass/fail
@pytest.mark.parametrize(
    "text,expected_count",
    [
        ("Paragraph one.\n\nParagraph two.\n\nParagraph three.", 3),
        ("Just one paragraph, no blank lines.", 1),
        ("A.\n\n\nB.", 2),      # extra blank lines: no empty chunks
        ("", 0),               # an empty file: no chunks at all
        ("  \n\n  ", 0),       # only spaces: same as empty
    ],
)
def test_chunk_by_paragraph_count(text, expected_count):
    chunks = chunk_by_paragraph(text)
    assert len(chunks) == expected_count
```

Expected output (`pytest chunking_unit_test_practice.py -q` , the short form — with `-v` , `pytest` puts each row's values into the test name, so those lines get very long):

```
..... [100%]
5 passed in 0.01s
```

The last three rows pass because Doc08's chunker strips each piece and skips empty ones. If yours returned `[]` for an empty string, the `("", 0)` row would fail — decide which behavior you want, then make the code match it. Don't just delete the row.

Difference from Basic: Approach 1 checks content as well as count, and adds a second, clearly named test for the single-paragraph case. Approach 2 covers the same ground and more through one `parametrize` table — including the empty and extra-blank-line inputs real documents produce — so each row is reported as its own pass/fail without a new function per case.

Which one should you actually write? Approach 1's content checks belong in every chunker test — a count alone proves too little. Add Approach 2's table as soon as you have more than two or

three inputs to check; it's the cheapest way to make sure the messy cases stay tested. The Build Task's `test_unit_layer.py` uses both.